

UDESC

Departamento de Ciência da Computação

Teoria dos Grafos: Tarefa 1

**Representação Computacional de um grafo não
direcionado através de uma Lista de Adjacências**

Alunos: Heitor Linhares Caetano e Deivid Veras Lourenço

Professor: Gilmário Barbosa dos Santos

Maio
2026

Conteúdo

1	Objetivo do Trabalho	1
2	Solução fornecida e sobre o projeto	2
2.1	Leitura do CSV	2
2.2	Grau mínimo e máximo	3
2.3	Quantidade de laços e arestas múltiplas	3
2.4	Determinação de componentes conexos	5
3	Sobre o ambiente de desenvolvimento e compilação	6
4	Análise dos Resultados	7
4.1	Resultados obtidos	8
5	Considerações Finais	11

1 Objetivo do Trabalho

A atividade consistiu em implementar um grafo através da representação computacional *Lista de Adjacências*, carregado a partir de uma base de dados no formato CSV.

A Lista de Adjacências é uma estrutura de dados composta por um vetor fixo de ponteiros para *nós* (unidade básica de uma lista encadeada). No caso do trabalho, cada índice do vetor representa um vértice diferente ($v_1, v_2, \dots$), e cada lista associada a um índice armazena as adjacências do vértice correspondente.

Vértice	Adjacentes
1	2, 3, 5
2	1, 4
3	1
4	2, 5
5	1, 4

Figura 1: Representação de uma lista de adjacências. O vértice 1 é conectado a 2, 3 e 5.

Foram implementadas funções que fornecem os graus máximo e mínimo do grafo, funções responsáveis por verificar se o grafo é simples ou multigrafo, bem como contabilizar laços e arestas múltiplas. Como função principal, foi implementado um procedimento que determina quantos componentes conexos existem no grafo, além dos tamanhos de cada um deles.

2 Solução fornecida e sobre o projeto

O projeto foi administrado em um [repositório](#) do GitHub e separado em diferentes arquivos, buscando melhor organização do código. Também usamos a *keyword static* em funções que não precisam ser acessadas fora do próprio arquivo. Isso, somado ao uso de diferentes arquivos, contribui para a aplicação das boas práticas de modularização e encapsulamento.

Além disso, utilizamos *unsigned ints* para a representação de vértices. Como índices de vetores (usados para acesso às listas) não assumem valores negativos, o uso de tipos sem sinal permite aproveitar todos os bits do inteiro, aumentando o maior índice representável sem precisar gastar mais memória.

A seguir vai uma explicação simples sobre cada exigência:

2.1 Leitura do CSV

Primeiramente, é importante ressaltar que o arquivo enviado não é exatamente um CSV, pois os valores ali estão separados por espaços, e não por vírgulas. Isso não é um grande problema, visto que alterar o código para suportar esse "SSV" seria trivial.

Mesmo assim, decidimos alterar o arquivo da base de dados de modo a transformá-lo em um CSV. Tratou-se de uma tarefa simples, necessitando apenas dos seguintes comandos em *Bash*:

```
$ tr ' ' ',' < teste2.csv > tmp.csv  
$ mv -f tmp.csv teste2.csv
```

Figura 2: O comando *tr* está trocando todas as aparições do caractere espaço por uma vírgula.

Antes de começarmos a ler os valores, precisamos determinar o tamanho do vetor fixo que abrigará as adjacências. Para isso, foi criada uma função que corre por todos os números e acha o maior dentre eles. Esse número (+1) é então utilizado como tamanho do vetor, representando o maior índice existente no grafo.

Um possível problema dessa implementação é a eventualidade de não existir algum vértice entre 1 e o maior número, o que poderia significar que ele simplesmente não existe, mas aqui é interpretado como um vértice de grau nulo. Em uma conversa com o professor, o mesmo declarou nossa solução como válida.

Além disso, é importante notar que há um erro no arquivo CSV, onde as linhas 499 e 500 apresentam apenas um vértice, e não um par. O modo de

lidar com essa parte pode alterar significativamente a interpretação do grafo pelo *software* e por nós. No nosso caso, decidimos fazer com que o código ignorasse linhas que não contêm exatamente 2 valores.

Ao ler o arquivo, como estamos tratando de grafos não direcionados, para cada par de valores a , b precisamos adicionar um novo *nó* às listas de adjacências de ambos os vértices. No caso em que $a = b$, essa inserção é realizada apenas uma vez, a fim de evitar a duplicação de laços.

2.2 Grau mínimo e máximo

Aqui a implementação foi relativamente direta. Nos dois casos, todas as listas têm seus tamanhos contados através de um laço de repetição e o menor/menor número é retornado no final.

2.3 Quantidade de laços e arestas múltiplas

A existência de laços e/ou arestas múltiplas em um grafo confirma que este é um multigrafo. Existem várias abordagens possíveis para a contagem de valores repetidos, e a complexidade de cada uma foi considerada para a nossa escolha final.

Inicialmente, a ideia foi de fazer a contagem através de um vetor de frequência. Nessa estratégia, para cada vértice adjacente na lista, incrementa-se a posição correspondente em um vetor auxiliar. Ao final da leitura, valores maiores que 1 significam a presença de múltiplas arestas. Isso nos permite contabilizar duplicatas e laços de forma direta, através do vetor de frequências.

Essa abordagem tem, para cada vértice, complexidade de tempo $O(g+V)$, onde g é o grau do vértice e V o número total de vértices do grafo. Isso porque um vetor de tamanho V era inicializado e percorrido em cada vértice, o que significa que a complexidade de espaço também era $O(V)$. Em grafos esparsos (como o analisado), isso significa que ocorre um desperdício de memória, pois grande parte do vetor nem chega a ser usada.

Como o procedimento é executado para cada vértice, a complexidade total da função era $O(V^2 + E)$, onde E é o número de arestas. Assim, o desempenho dependia fortemente da quantidade total de vértices do grafo, e não apenas das arestas existentes.

A segunda abordagem, que acabou sendo escolhida, consiste em copiar os valores da lista para um vetor auxiliar e, em seguida, ordená-lo. Após a ordenação, basta percorrer o vetor uma única vez verificando elementos consecutivos iguais para identificar arestas múltiplas.

Essa implementação possui complexidade temporal $O(g \log g)$ para cada vértice. Mesmo sendo *assintoticamente* menos eficiente do que a estratégia com vetor de frequência, essa solução tira a dependência do número total de vértices, sendo mais *bem comportada* para grafos dos mais diversos tipos.

Além disso, a complexidade espacial passa a ser $O(g)$, já que o vetor auxiliar possui tamanho proporcional apenas ao grau do vértice analisado. Em grafos esparsos, isso reduz significativamente o consumo de memória. Com o procedimento sendo executado para cada vértice, a complexidade temporal total da função é de $O(\sum_{i=1}^V g_i \log g_i)$, com limite superior de $O(E \log V)$.

Vale a pena ressaltar que os laços recebem tratamento diferente dos outros vértices repetidos. Enquanto arestas múltiplas aparecem duas vezes na lista de adjacência, o que faz com que se exija uma divisão por dois no final, laços são armazenados apenas uma vez, sendo contabilizados separadamente.

2.4 Determinação de componentes conexos

Um grafo é dito conexo quando existe caminho entre qualquer par de seus vértices, ou seja, todo vértice está ao alcance de todos os demais. Quando isso não ocorre, o grafo é desconexo e pode ser dividido em partes menores, cada uma conexa. Essas partes são chamadas de componentes conexos.

Um componente conexo é um subgrafo conexo maximal, ou seja, o maior subgrafo possível que mantém a conectividade entre seus vértices.

Para identificarmos esses componentes conexos, inicialmente usamos como ideia a busca em largura (BFS), porém durante a análise da implementação, percebemos que a busca em profundidade (DFS), utilizando recursão, tornaria o código mais simples.

Usamos como base uma pequena modificação do pseudocódigo de DFS (busca em profundidade) presente nos *slides* das aulas de Teoria dos Grafos, elaborados pelo professor:

1 DFS

```
1: procedure DFS(G, v, M, marca)
2:   marca[v] = 1
3:   acumulador = 1
4:   for w adjacente a v do
5:     if marca[w]=0 then
6:       acumulador += DFS(G, w, M, marca)
7:     end if
8:   end for
9:   return acumulador
10: end procedure
```

No código, o funcionamento ocorre da seguinte forma: percorrem-se todos os vértices do grafo e, sempre que um vértice ainda não visitado é encontrado, inicia-se uma nova busca a partir dele. Essa busca visita recursivamente todos os vértices alcançáveis a partir do vértice inicial, marcando-os como visitados e contabilizando quantos pertencem àquele grupo.

Portanto, cada vez que uma nova busca precisa ser iniciada, significa que um novo componente conexo foi descoberto.

Mesmo preferindo e adotando como padrão a DFS, como a implementação da BFS já estava feita, decidimos fazer dela um algoritmo opcional, selecionável através de um argumento extra no comando de compilação (Instruções no próximo capítulo). A busca em largura tem a vantagem de não usar recursão, o que a torna mais adequada para grafos grandes.

3 Sobre o ambiente de desenvolvimento e compilação

Como explicado anteriormente, um repositório de código-fonte foi instalado no GitHub para uma melhor colaboração.

Como sistema operacional, foram utilizados tanto o Windows (Deivid) quanto GNU/Linux (Heitor). Mesmo com ambientes diferentes, miramos em ter uma *codebase* compatível, sem usar funções não portáteis, como *system()*.

Sobre os ambientes de desenvolvimento, Deivid usou o [Visual Studio Code](#), enquanto Heitor usou o editor de texto [Helix](#). Como este não usou uma *IDE* completa, algumas ferramentas de linha de comando foram usadas para emular funcionalidades comuns em outros ambientes, especificamente [cproto](#) para a geração automática de protótipos e [astyle](#) como formatador de código.

Como as demais ferramentas, também foi utilizada a linha de comando para compilar o projeto, em que foi adotado o compilador de C do *GNU (GCC)*.

Pelo fato de algumas funções terem sido implementadas com *recursão de cauda*, é recomendável compilar o código com nível de otimização 2 ou superior. Dessa forma, o compilador pode aplicar o que é chamado de *tail call optimization* (TCO), gerando código assembly equivalente a versões iterativas dessas funções.

Assim, o comando para compilar em ambientes GNU/Linux é:

```
$ gcc ls_adj.c csv.c main.c -O2 -o grafo
```

e para iniciar o programa, simplesmente execute:

```
$ ./grafo
```

Para utilizar o algoritmo de busca em largura (BFS) em vez da busca em profundidade, basta adicionar o argumento `-DBFS` ao comando de compilação:

```
$ gcc ls_adj.c csv.c main.c -O2 -o grafo -DBFS
```


4 Análise dos Resultados

Primeiramente, podemos tentar visualizar a quantidade de componentes conexos através de um programa dedicado a isso. Inicialmente, usamos o *script* escrito em *python*, e distribuído pelo professor. Seguem os resultados:

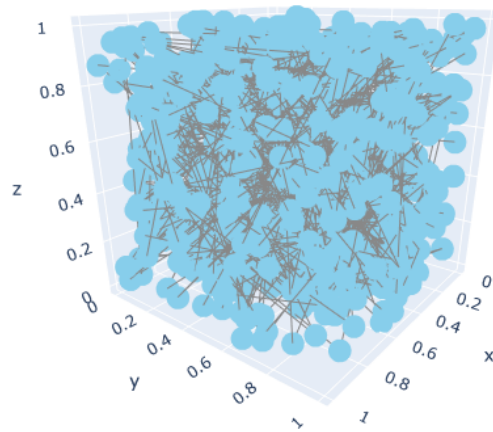


Figura 3: Visualização 3D do grafo.

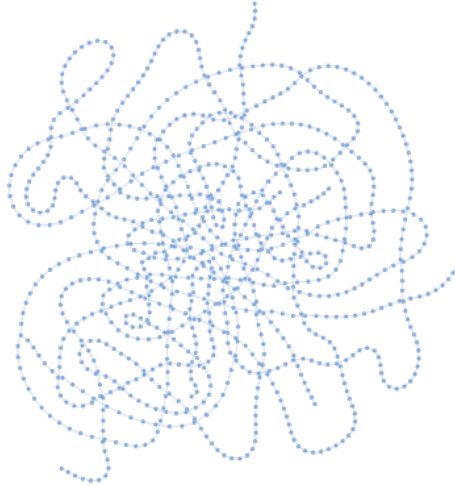


Figura 4: Visualização 2D do grafo.

Por meio dessas imagens, é possível concluir que o grafo é grande demais para ser analisado a olho nu, pelo menos através da representação visual dada

pelo script.

Em busca de um *software* de visualização de grafos mais poderoso, achamos o [GraphViz](#), que possui algoritmos específicos para grandes grafos. Tomando proveito de algumas configurações do programa e usando o algoritmo [sfdp](#), obtivemos esse resultado:

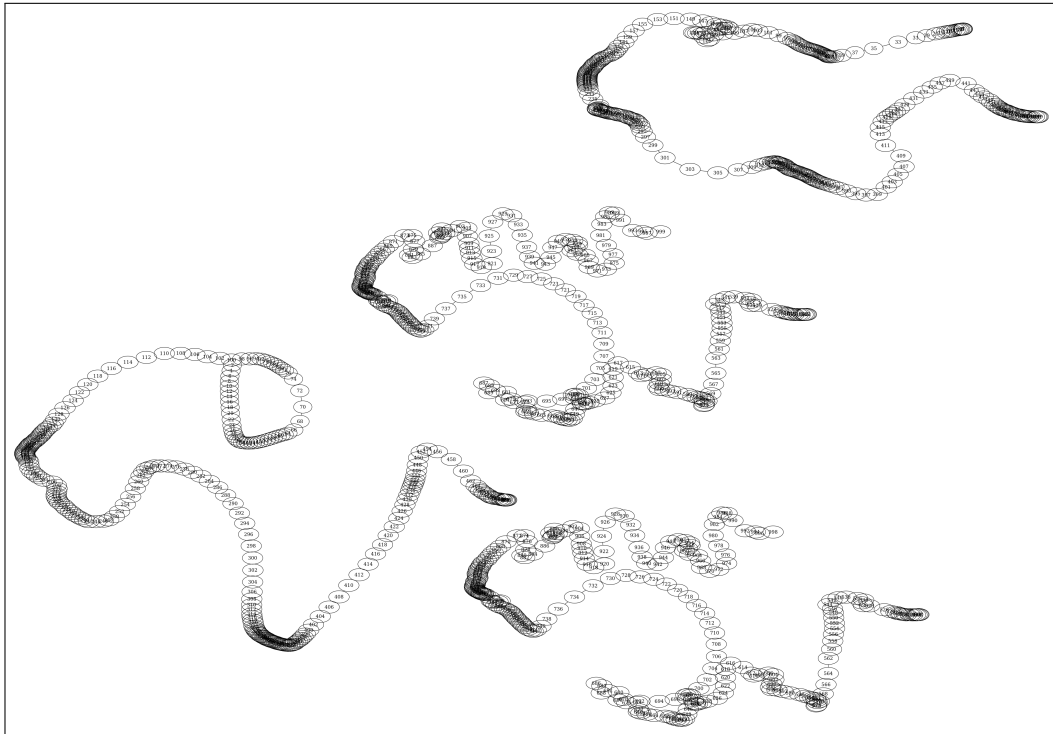


Figura 5: Visualização do grafo através do *software* Graphviz.

A partir da imagem gerada conseguimos ter uma ideia melhor do que esperar como *output* do nosso projeto. É possível visualizar claramente 4 componentes conexos, sendo que todos aparentam ser caminhos, exceto o mais à esquerda, que possui um ciclo como subgrafo. Assim, aparentemente o maior grau do grafo total seja 3 (seria o do vértice que faz a conexão entre o subgrafo caminho e ciclo).

4.1 Resultados obtidos

A partir da execução das funcionalidades implementadas, foi possível observar algumas propriedades relevantes do grafo construído a partir do arquivo CSV, que possui 999 vértices identificados.

O grau máximo encontrado foi 3, confirmando o que tínhamos deduzido anteriormente. Já a função de grau mínimo encontrou 1 como sendo o menor valor, o que nos mostra não haver vértices isolados no grafo. Além disso, fazendo uma análise da imagem, podemos entender que os vértices de grau 1 são as extremidades dos subgrafos do tipo caminho.

Na verificação de multigrafo, o sistema analisa a existência de laços e arestas múltiplas. Essa etapa diferencia grafos simples de estruturas mais complexas. Como não foram encontrados laços nem arestas múltiplas, o grafo foi classificado como simples.

Na análise de componentes conexos, foram identificados quatro componentes, também confirmando nossa análise prévia. Isso significa que não existe um único caminho que conecta todos os vértices da estrutura. Também conseguimos extrair o tamanho de cada componente.

Através dos resultados obtidos e com a ajuda do *software* [gnuplot](#), criamos um histograma:

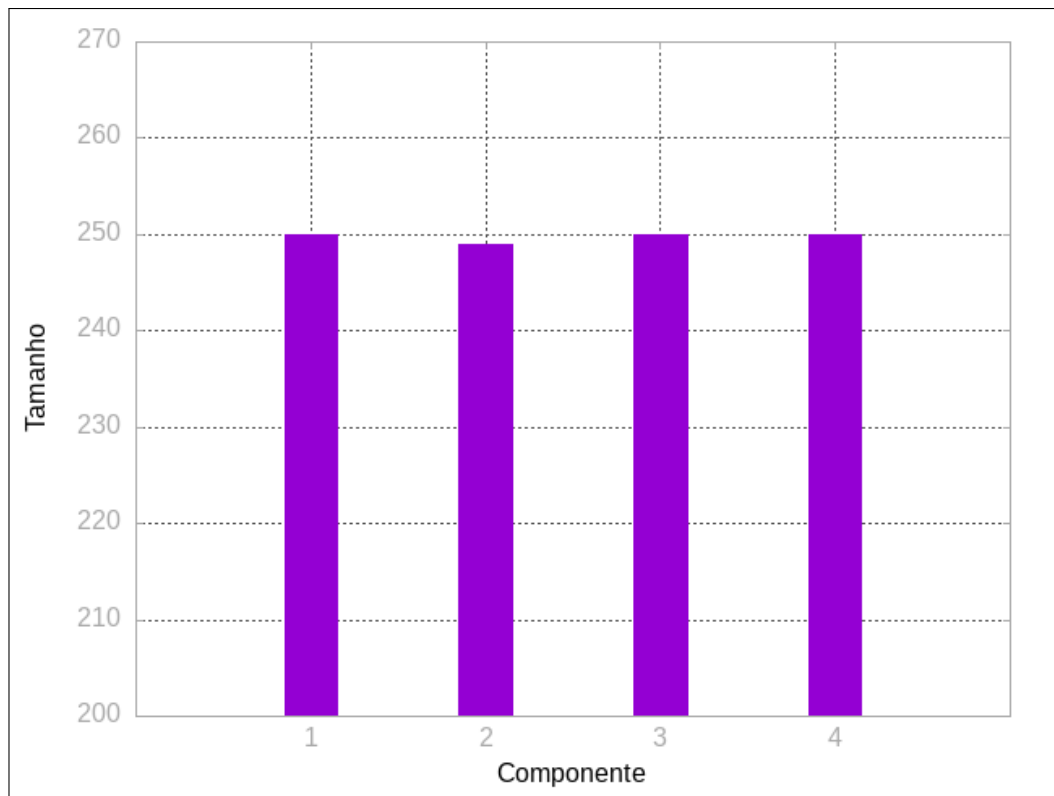


Figura 6: Histograma.

Aqui é possível ver que os quatro componentes possuem praticamente

o mesmo tamanho, com apenas um apresentando diferença mínima dos demais. Mais especificamente, três dos componentes possuem exatamente 250 vértices, enquanto o outro é composto por 249.

5 Considerações Finais

Com o desenvolvimento deste trabalho, foi possível aplicar na prática conceitos fundamentais de Teoria dos Grafos, como grau de vértices, classificação entre grafo simples e multigrafo, além da identificação de componentes conexos.

A implementação permitiu compreender melhor o uso de algoritmos de busca em grafos, tanto a busca em profundidade (DFS) quanto a busca em largura (BFS), sendo que a DFS se mostrou uma solução mais simples e *inteligível*.

Por meio dos resultados obtidos, foi possível verificar que a estrutura analisada é um grafo simples e desconexo, dividido em quatro componentes principais. Dessa forma, o trabalho contribuiu tanto para o entendimento teórico quanto para a aplicação prática dos algoritmos estudados na disciplina.